Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота № 1
з дисципліни
«Системне програмування»

ВИКОНАВ:
студент IV курсу ФІОТ
групи ІО-__
Прізвище Ім’я По батькові
Залікова № ____
Варіант – __

ПЕРЕВІРИВ:
Гордієнко Н. Ю.

Київ – 2026

Тема: Налаштування середовища розробки та bring-up STM32-платформи.

Мета роботи: отримати відтворюване середовище для розробки прошивок STM32 і пройти повний шлях від порожньої теки до прошивки, яка відповідає по UART: зібрати тулчейн у Docker, згенерувати скелет проєкту в STM32CubeMX, зібрати його однією командою та запустити в симуляторі Wokwi.

На старті задача виглядала як «встановити програми і побачити tick у терміналі». У процесі вона уточнилася: потрібно було ще підібрати плату, яку реально підтримує симулятор, узгодити піни мікроконтролера з периферією (UART, I2C, ADC, PWM, GPIO) і зі схемою в Wokwi, перевірити, що шина I2C справді «жива», а також визначити тему проєкту та описати вимоги до нього в PRD.

Форма виконання: В (симулятор Wokwi), індивідуально. Плата — NUCLEO-C031C6 (STM32C031C6, Cortex-M0+), оскільки плати серії F4 з методички у Wokwi не симулюються. Програма написана на C з використанням STM32 HAL.

Виконання лабораторної роботи

Крок 1. Налаштування build-системи

Збірка виконується в Docker-контейнері в середовищі WSL2 (Ubuntu 24.04). У теці docker/ створено Dockerfile на основі ubuntu:22.04 з arm-none-eabi-gcc, newlib і srecord; образ зібрано командою docker build -t stm32-build docker/.

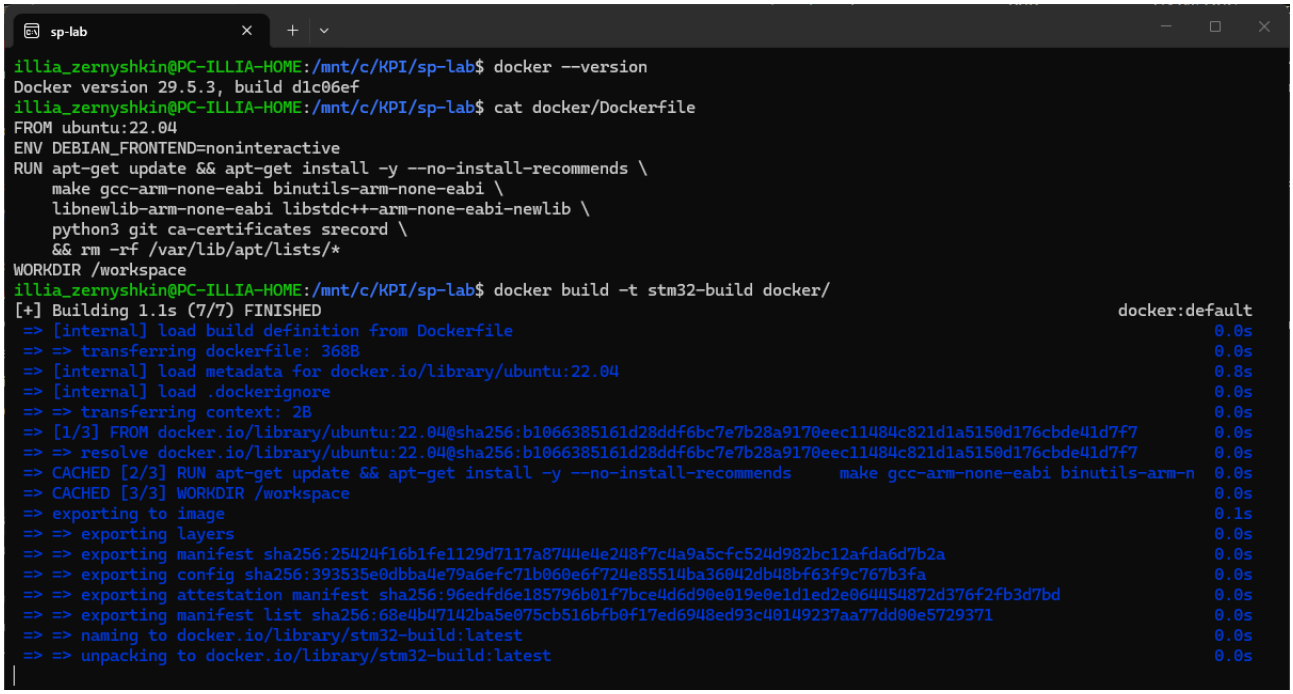


Рис. 1. Dockerfile і збірка образу stm32-build

Кореневий Makefile запускає збірку в контейнері. Ідентифікатори користувача передаються через $(shell id -u):$(shell id -g), щоб артефакти збірки не належали root. Додано цілі image, build, clean, flash і monitor.

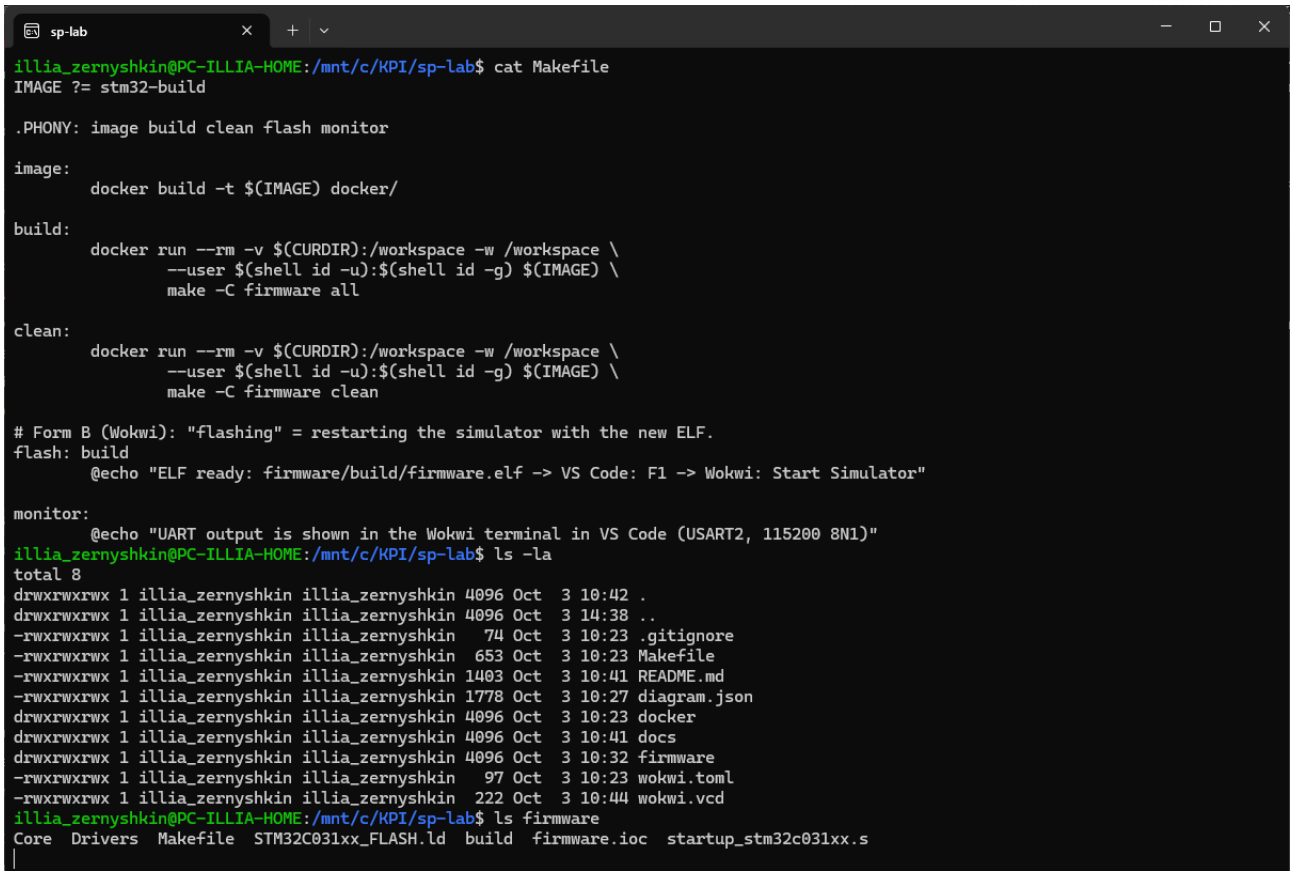


Рис. 2. Кореневий Makefile і структура проєкту

Крок 2. Створення скелету проєкту через CubeMX

Проєкт створено в STM32CubeMX 6.18.1 через Board Selector для плати NUCLEO-C031C6. Шар BSP (Human Machine Interface) вимкнено, щоб UART працював через звичайний HAL-дескриптор huart2. Увімкнено таку периферію:

- USART2 → Asynchronous, 115200 8N1 (PA2 — TX, PA3 — RX);

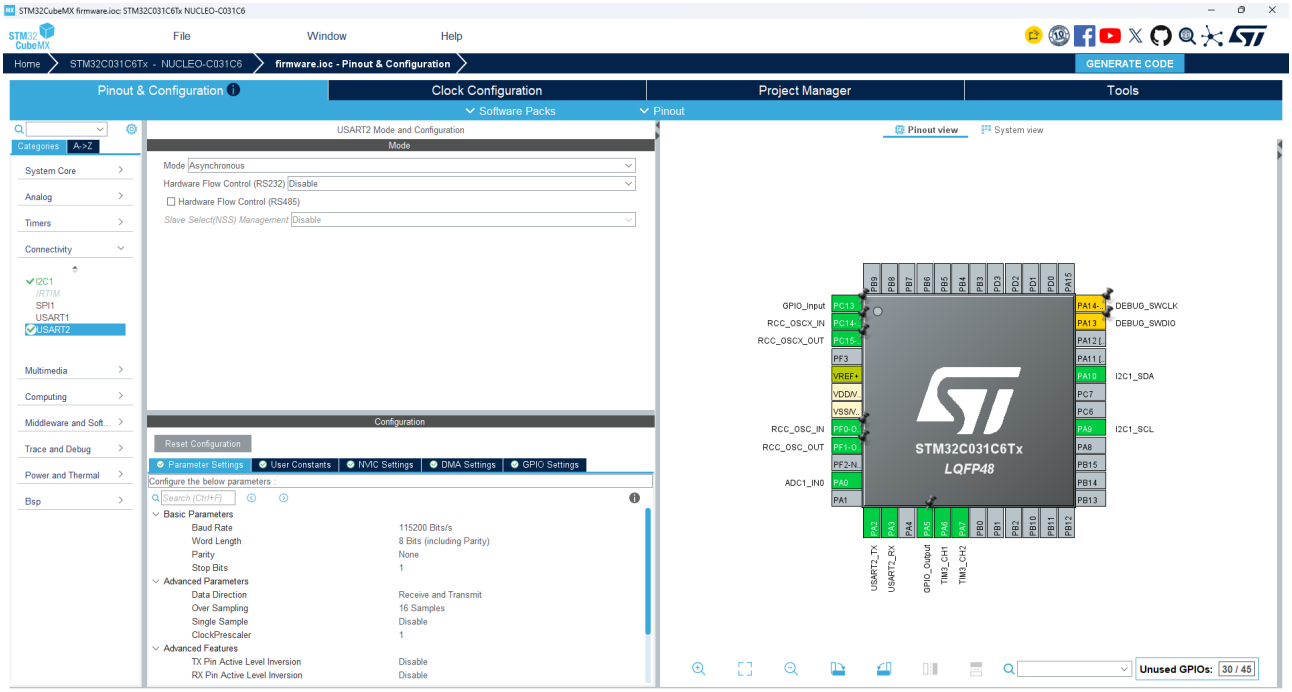


Рис. 3. USART2

- I2C1 → Standard Mode, 100 кГц (PA9 — SCL, PA10 — SDA) для MPU-6050;

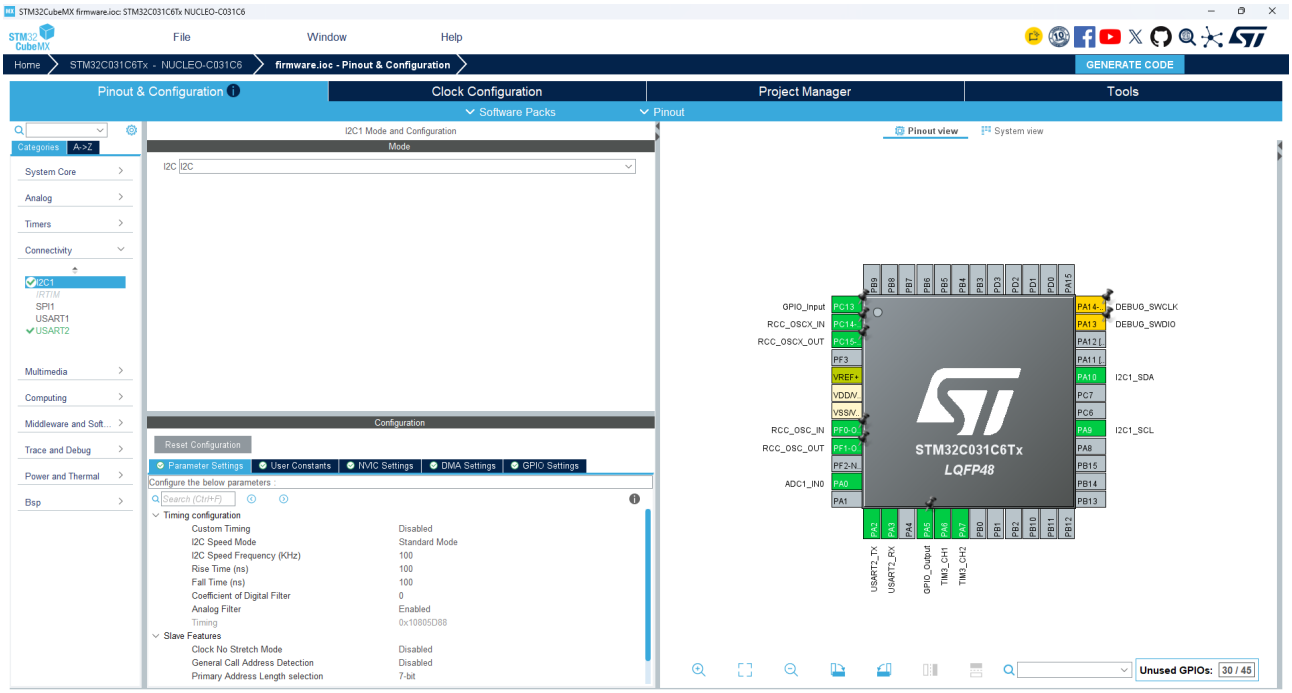


Рис. 4. I2C1

- TIM3 CH1, CH2 → PWM Generation (PA6, PA7) для двох сервоприводів;

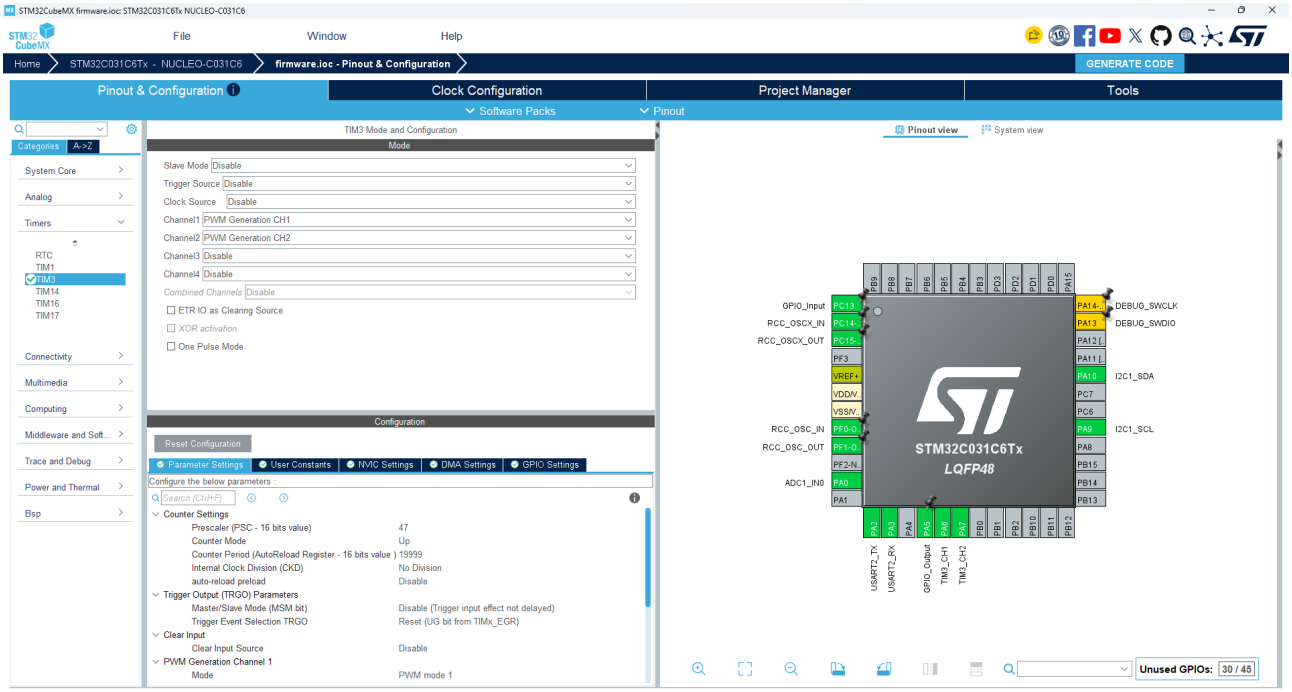


Рис. 5. TIM3, PWM

Частота таймера: 48 МГц / (47 + 1) = 1 МГц, тобто 1 такт = 1 мкс. Counter Period = 19999 дає період 20 мс (50 Гц), потрібний сервоприводам. Тривалість імпульсу 1500 мкс (центральне положення) задається в коді через __HAL_TIM_SET_COMPARE.

- ADC1 IN0 (PA0), 12 біт — потенціометр;

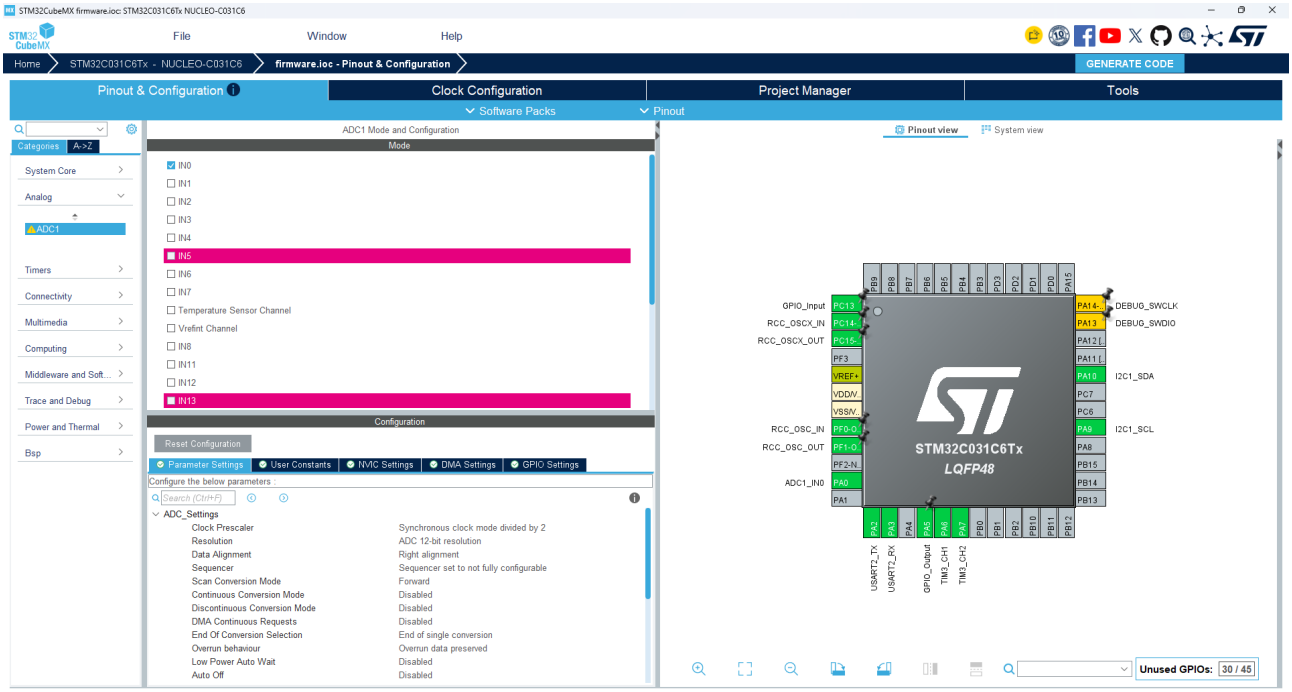


Рис. 6. ADC1

- GPIO: PA5 — вихід (світлодіод LD4), PC13 — вхід з підтяжкою pull-up (кнопка).

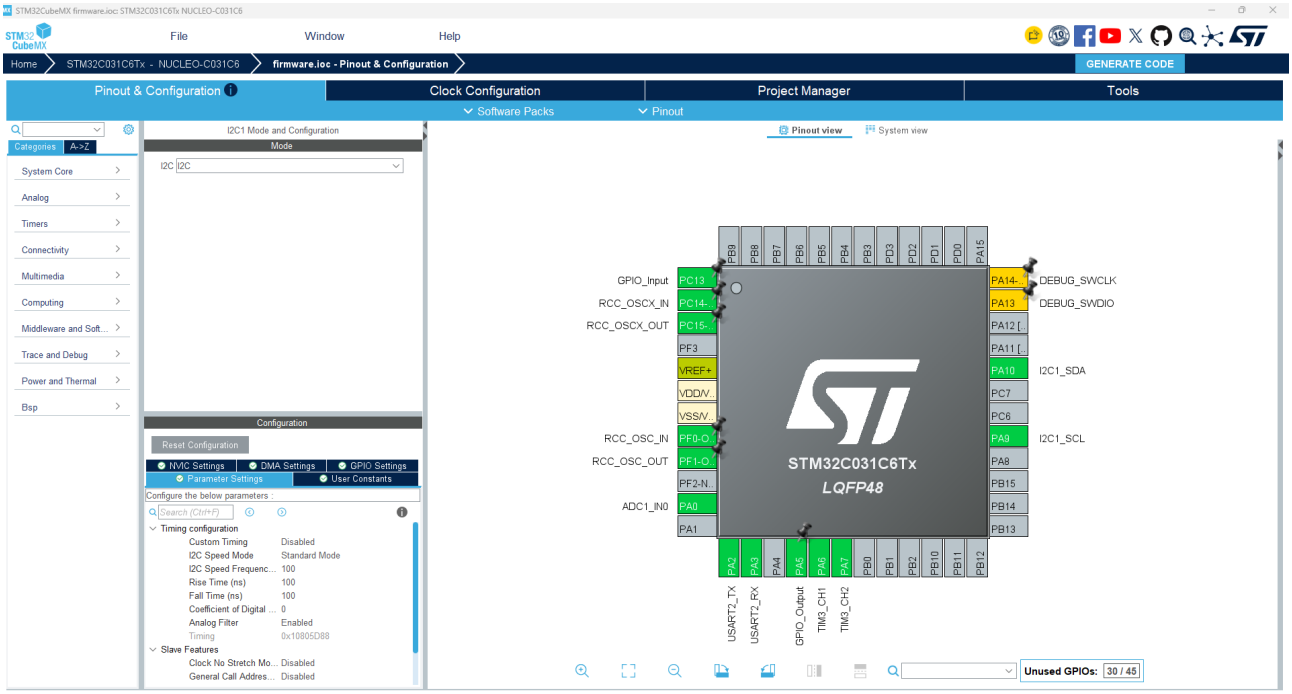


Рис. 7. Підсумкова розводка пінів

Джерело тактування — внутрішній генератор HSI48: SYSCLK = HCLK = PCLK = 48 МГц, усі дільники /1.

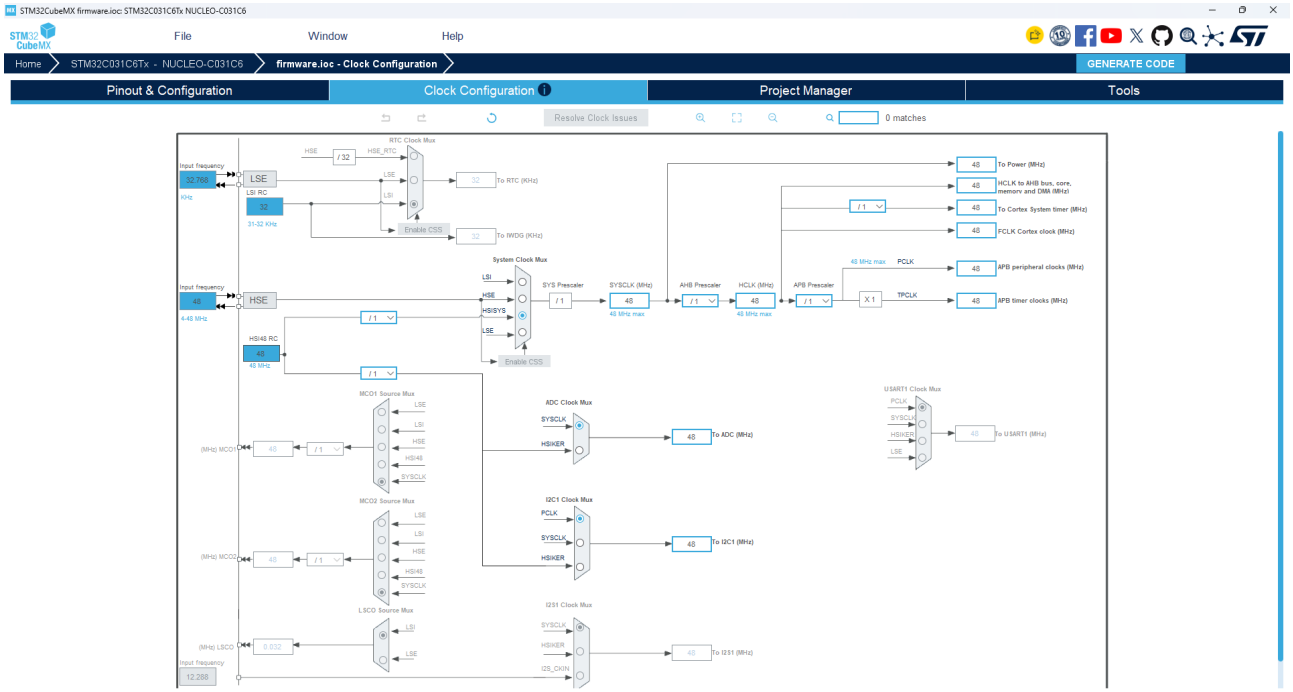


Рис. 8. Clock Configuration

У Project Manager обрано Toolchain / IDE: Makefile, ім’я проєкту firmware, розташування — корінь репозиторію. Після Generate Code у теці firmware з’явилися Core/, Drivers/, Makefile, startup_stm32c031xx.s, лінкер-скрипт STM32C031xx_FLASH.ld і firmware.ioc.

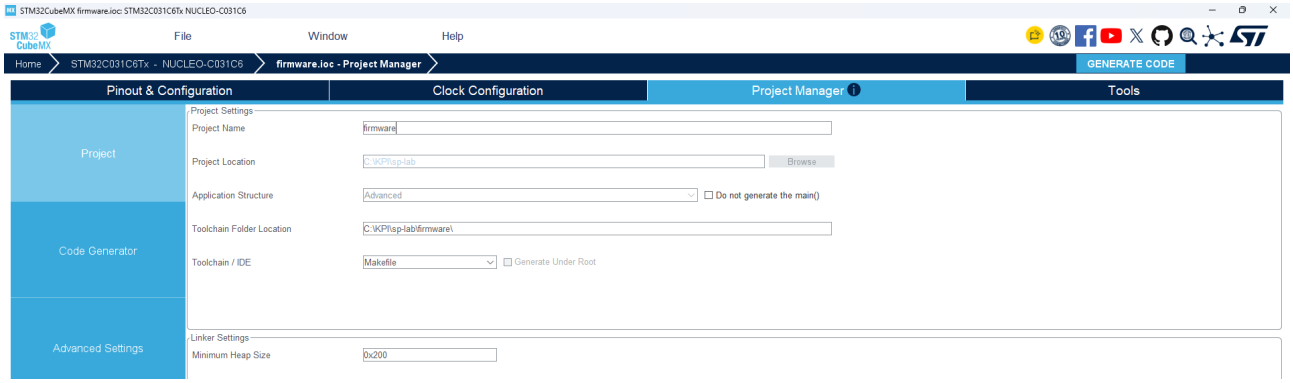


Рис. 9. Project Manager

Згенерований firmware/Makefile використовує прапорці -mcpu=cortex-m0plus -mthumb (без FPU), тому адаптувати їх вручну не довелося. Команда make build збирає firmware.elf, .hex і .bin; розмір прошивки — 18 820 байт коду при 32 КБ flash.

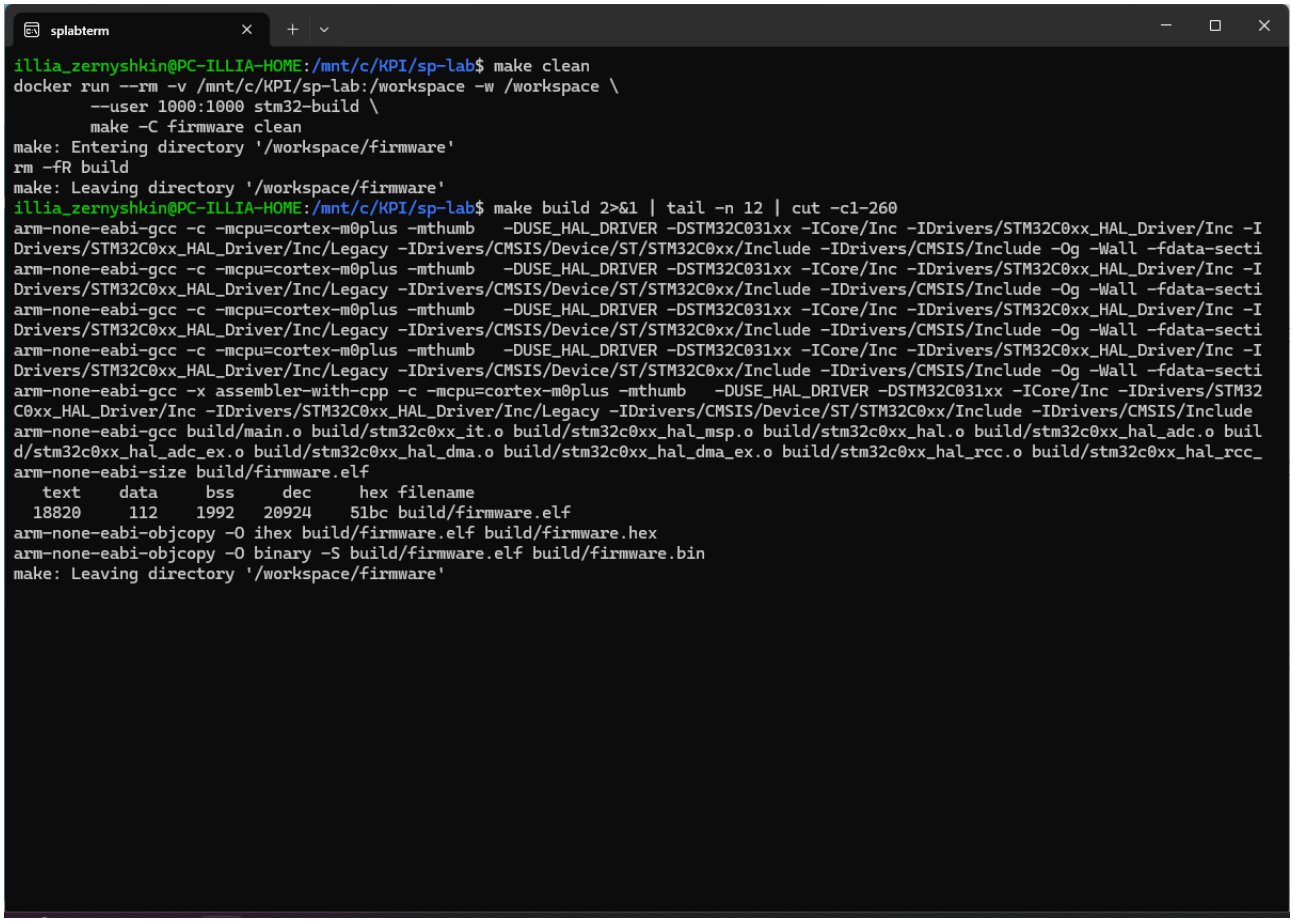


Рис. 10. Збірка проєкту в контейнері

Крок 3. Прошивка (форма B)

Встановлено розширення Wokwi Simulator для VS Code та активовано ліцензію.

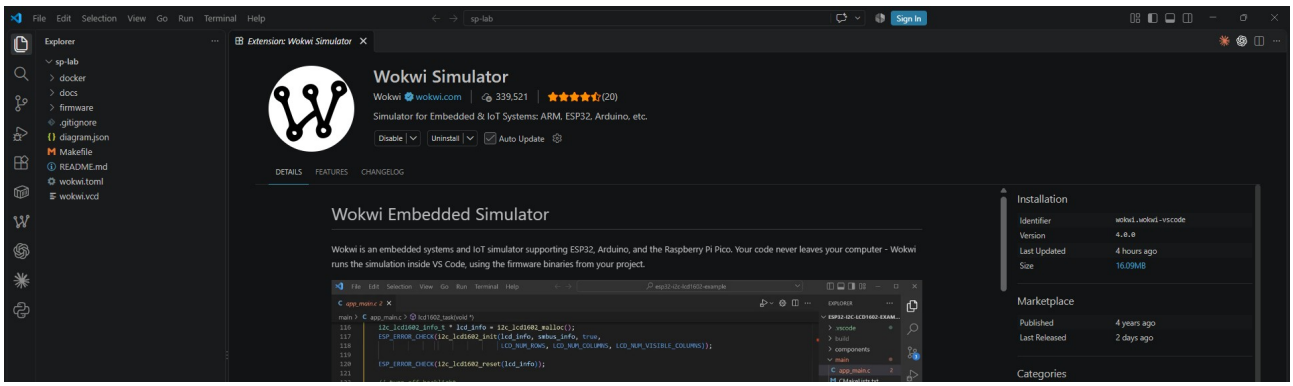


Рис. 11. Розширення Wokwi Simulator

У корені проєкту створено wokwi.toml, який указує на firmware/build/firmware.elf, і diagram.json зі схемою: плата NUCLEO-C031C6, MPU-6050 (I2C), потенціометр (ADC), кнопка (GPIO), два сервоприводи (PWM) і логічний аналізатор на лініях SCL/SDA. «Прошивка» у формі B — це перезапуск симулятора з новим ELF-файлом.

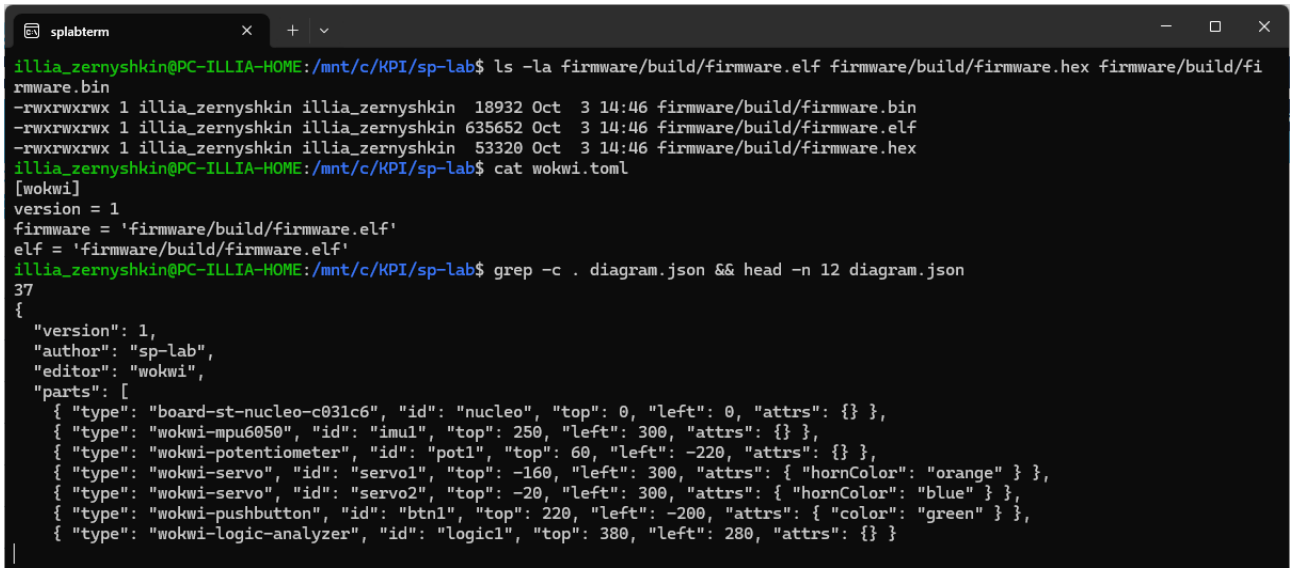


Рис. 12. Артефакти збірки, wokwi.toml і початок diagram.json

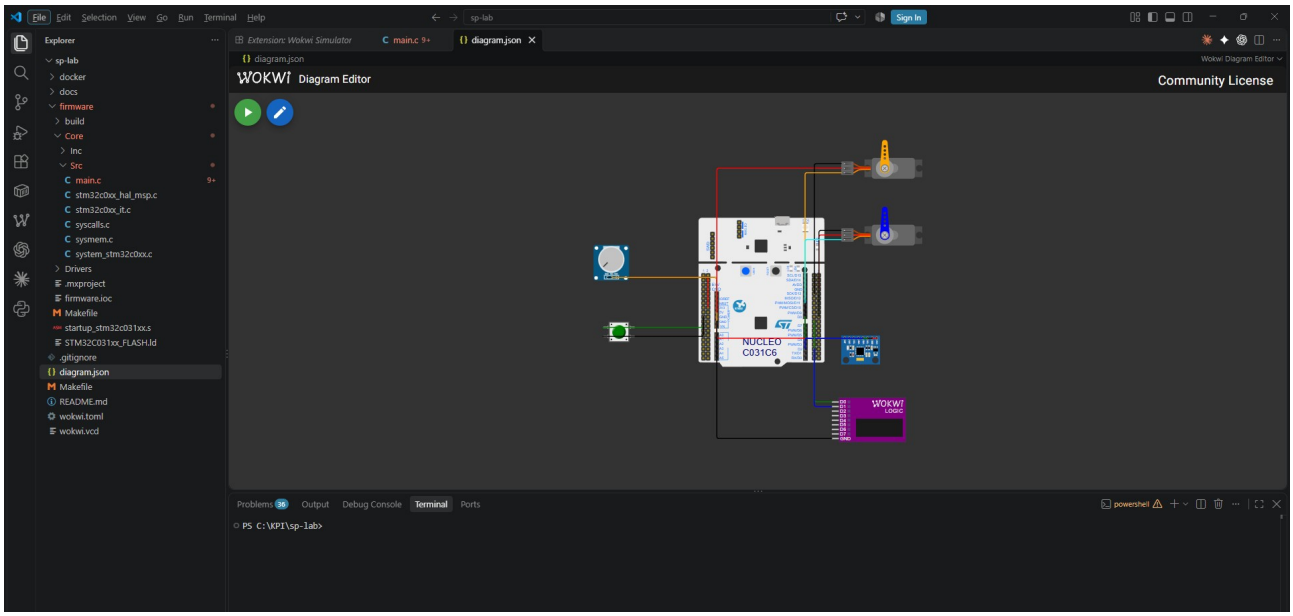


Рис. 13. Схема в редакторі Wokwi

Крок 4. Перша відповідь по UART

У main.c додано хук _write(), який перенаправляє printf у USART2, привітання після ініціалізації та щосекундне повідомлення tick зі значенням ADC і станом кнопки. У циклі також перемикається світлодіод LD4.

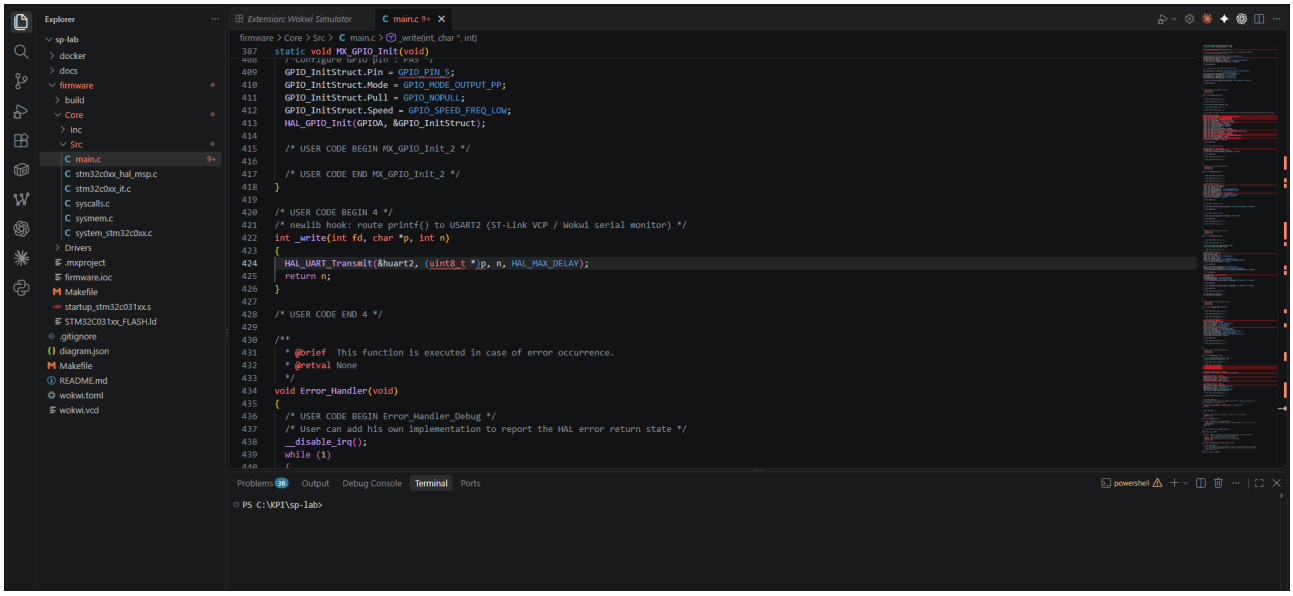


Рис. 14. Хук _write() у main.c

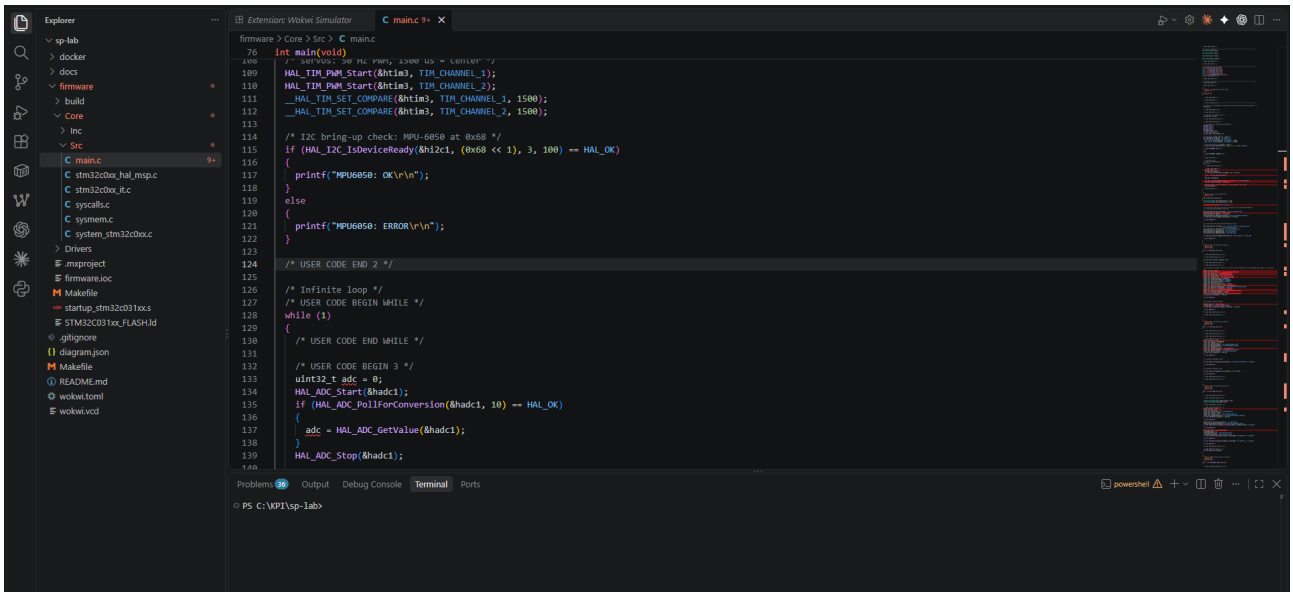


Рис. 15. Ініціалізація та основний цикл

Після запуску симулятора в терміналі Wokwi з’являється hello from STM32, далі щосекунди — tick. Після повороту потенціометра значення adc змінюється (468), після натискання кнопки btn стає 1 — отже, ADC і вхід GPIO працюють.

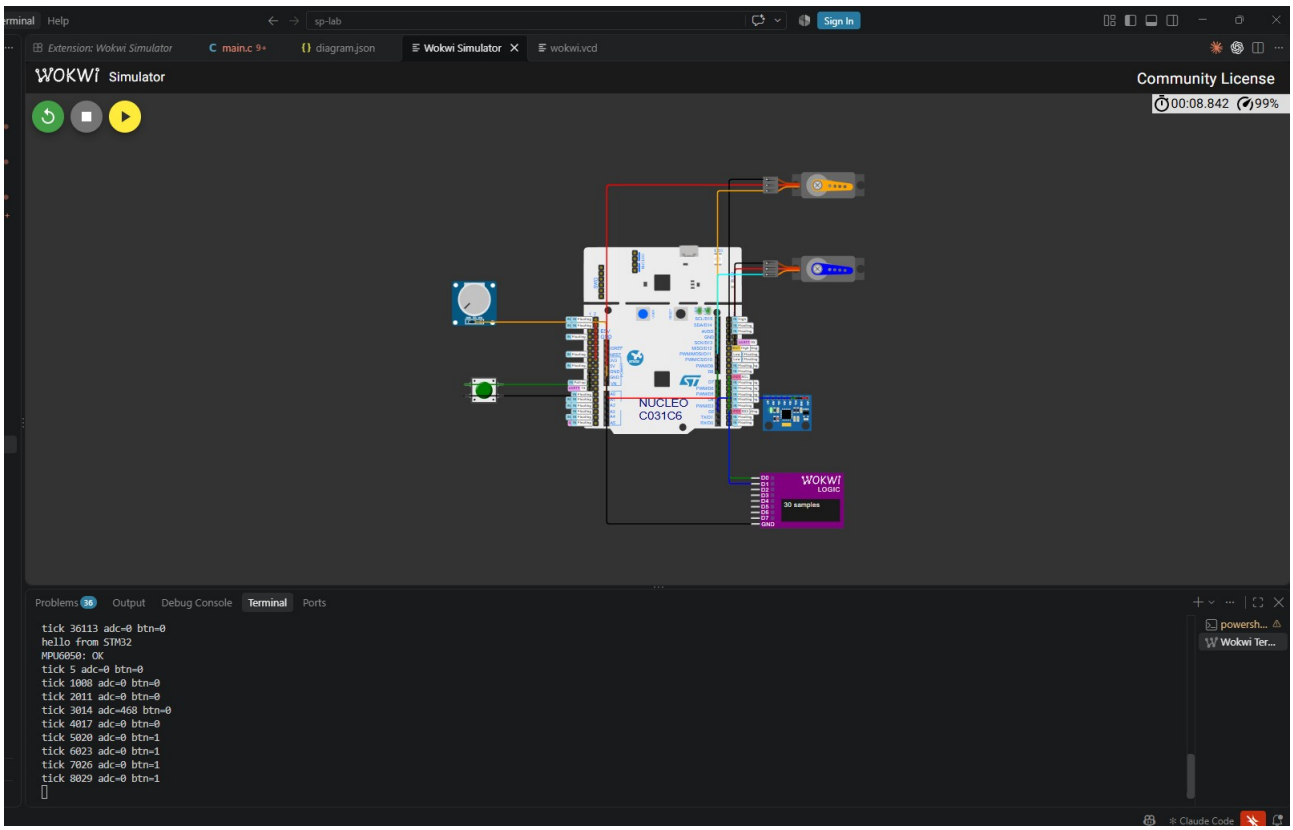


Рис. 16. Симулятор Wokwi і вивід USART2

Крок 5. Дослідження компонентів

Опис компонентів (інтерфейс, адреса, живлення, піни, регістри) і таблиця розводки наведені у файлі docs/hardware-components.md репозиторію.

Крок 6. Генерація коду для інтерфейсів

STM32CubeMX згенерував функції MX_USART2_UART_Init(), MX_I2C1_Init(), MX_ADC1_Init(), MX_TIM3_Init() і MX_GPIO_Init(). Параметри інтерфейсів:

- USART2: PA2 — TX, PA3 — RX, 115200 біт/с, 8 біт даних, 1 стоп-біт, без парності.
- I2C1: PA9 — SCL, PA10 — SDA, 100 кГц, 7-бітна адресація.
- ADC1: PA0 (IN0), 12 біт, програмний запуск перетворення.
- TIM3 PWM: PA6 — CH1, PA7 — CH2, 50 Гц, період 20 000 мкс.
- GPIO: PA5 — цифровий вихід, PC13 — цифровий вхід з pull-up.

Перевірка. Для I2C виконано HAL_I2C_IsDeviceReady() за адресою 0x68. Функція повернула HAL_OK — у терміналі видно MPU6050: OK (рис. 16). До PA9 (D0) і PA10 (D1) під’єднано Wokwi Logic Analyzer. У записі wokwi.vcd лінія SCL перемикається кожні 5 мкс, тобто період становить 10 мкс, що відповідає 100 кГц. Отже, лінії I2C не «мертві».

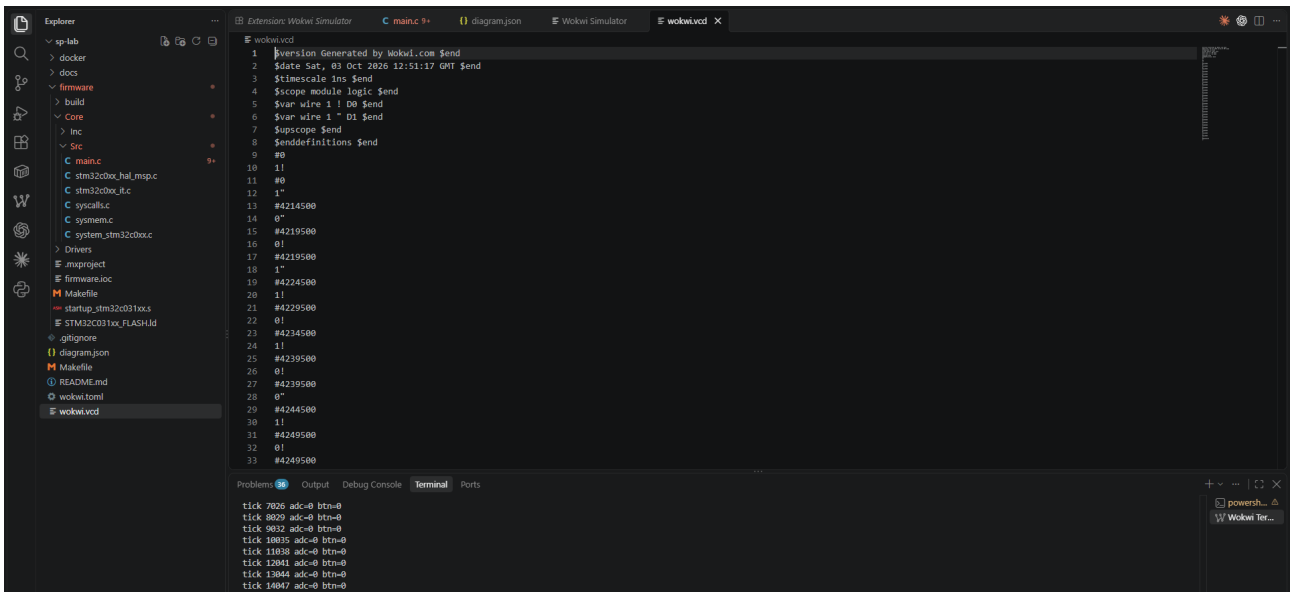


Рис. 17. Запис логічного аналізатора (wokwi.vcd): D0 — SCL, D1 — SDA

Крок 7. Тема проєкту та PRD

Обрано проєкт 6 — «Дзеркалення руху руки на сервоприводах» (Motion-Mirroring Teleoperator): MPU-6050 зчитує орієнтацію, два сервоприводи її повторюють, потенціометр задає чутливість, кнопка — калібрування та режим. PRD з функціональними вимогами, списком датчиків і планом тестування — у файлі docs/PRD.md.

Крок 8. Репозиторій

Приватний репозиторій: посилання на GitHub. Викладача додано як reviewer, робота здається через pull request у гілку main. Структура: docker/Dockerfile, firmware/, docs/PRD.md, docs/hardware-components.md, docs/screenshots/, Makefile, wokwi.toml, diagram.json, README.md, .gitignore.

Процес виконання

Короткий журнал виконання (03.10.2026):

1) Встановлено STM32CubeMX, обрано плату NUCLEO-C031C6 через Board Selector.
2) Налаштовано периферію: USART2, I2C1, TIM3 PWM, ADC1, GPIO; тактування переведено на HSI48.
3) Згенеровано код із Toolchain = Makefile.
4) Підготовлено Dockerfile і кореневий Makefile, зібрано образ stm32-build.
5) Виконано першу збірку make build, отримано firmware.elf.
6) Додано _write(), printf і перевірку MPU-6050 у main.c.
7) Створено wokwi.toml і diagram.json, запущено симулятор, отримано hello from STM32 і tick.
8) Отримано MPU6050: OK, логічним аналізатором перевірено SCL/SDA.

9) Підготовлено PRD, опис компонентів і README.

Проблеми та способи їх вирішення:

Проблема 1: У Wokwi немає плат серії F4
Що відбувалося: у методичці вказано NUCLEO-F411RE / F446RE, але в списку плат симулятора Wokwi їх немає.
Що виявилось причиною: Wokwi симулює лише окремі моделі STM32, зокрема NUCLEO-C031C6 і L031K6.
Як полагодили: обрано NUCLEO-C031C6 (більше RAM і пінів, ніж у L031K6). Через це змінилися прапорці компілятора (Cortex-M0+, без FPU), SPI2 на платі відсутній, а обсяг flash — 32 КБ, тому printf із float не використовується.
Git commit: посилання

Проблема 2: Плата не знаходилась у CubeMX
Що відбувалося: після введення NUCLEO-C031C6 список був порожній: «0 item (No Match)».
Що виявилось причиною: пошук виконувався на вкладці MCU/MPU Selector, яка шукає мікроконтролери, а не налагоджувальні плати.
Як полагодили: плату обрано на вкладці Board Selector.
Git commit: посилання

Проблема 3: Шар BSP займав піни UART
Що відбувалося: після ініціалізації периферії за замовчуванням піни PA2/PA3 були позначені як VCP_TX/VCP_RX, PA5 — LD4, PC13 — BUTTON; у коді не було б huart2 і MX_USART2_UART_Init(), на які розрахований хук _write().
Що виявилось причиною: для плат Nucleo CubeMX вмикає шар BSP (Human Machine Interface), який сам керує віртуальним COM-портом, світлодіодом і кнопкою.
Як полагодили: BSP вимкнено, USART2 увімкнено в режимі Asynchronous, PA5 і PC13 налаштовано як звичайні GPIO.
Git commit: посилання

Проблема 4: Джерело тактування HSE
Що відбувалося: у Clock Configuration за замовчуванням системним джерелом був HSE (зовнішній кварц 48 МГц).
Що виявилось причиною: це налаштування плати за замовчуванням; у симуляторі зовнішнього кварца може не бути, і тоді SystemClock_Config() зависла б в очікуванні готовності HSE. Саме зависання не спостерігалося — зміну зроблено завчасно.
Як полагодили: System Clock Mux переведено на HSISYS (HSI48), SYSCLK залишився 48 МГц.

Git commit: посилання

Проблема 5: Docker Desktop не запускався
Що відбувалося: під час запуску Docker Desktop з'являлося вікно «An unexpected error occurred» з помилкою ініціалізації Inference manager.
Що виявилось причиною: збій самого Docker Desktop на Windows; причину не досліджено.
Як полагодили: збірку виконано через Docker Engine, встановлений усередині WSL (Ubuntu 24.04); Docker Desktop для роботи не потрібен.
Git commit: посилання

Проблема 6: Команда Wokwi не знаходилась у VS Code
Що відбувалося: у рядку пошуку VS Code за словом Wokwi знаходився лише файл wokwi.toml, команди Start Simulator не було.
Що виявилось причиною: тека відкрилась у Restricted Mode, в якому розширення вимкнені; крім того, пошук ішов по файлах, а не по командах.
Як полагодили: теку додано до довірених (Trust), команду викликано через F1 → Wokwi: Start Simulator.
Git commit: посилання

Висновки:

Під час роботи отримано практичний досвід підготовки середовища для прошивок STM32: збірка в Docker-контейнері однією командою, генерація HAL-скелета в STM32CubeMX і запуск ELF-файлу в симуляторі Wokwi. Стало зрозуміло, як CubeMX прив'язує периферію до конкретних пінів через alternate functions, як шар BSP плати Nucleo конфліктує зі звичайною ініціалізацією USART2 і як дільник та період таймера визначають частоту PWM.

Перевірено роботу UART (printf через _write()), I2C (відповідь MPU-6050 і тактові імпульси 100 кГц на логічному аналізаторі), запуск PWM, читання ADC (потенціометр) і входу GPIO (кнопка).

У ЛР2 планується читати дані MPU-6050, обчислювати кути нахилу і керувати ними сервоприводами. З наявними знаннями варто було б одразу перевірити, які плати підтримує симулятор, і лише потім створювати проєкт у CubeMX.

Матеріали та посилання:

1) Wokwi Docs — STM32 Nucleo64 C031C6: https://docs.wokwi.com/parts/board-st-nucleo-c031c6
2) Wokwi Boards — опис пінів плати st-nucleo-c031c6: https://github.com/wokwi/wokwi-boards

3) Wokwi for VS Code: https://marketplace.visualstudio.com/items?itemName=wokwi.wokwi-vscode
4) STM32CubeMX: https://www.st.com/en/development-tools/stm32cubemx.html
5) MPU-6050 — сторінка продукту та документація TDK InvenSense: https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/
6) Методичні вказівки до лабораторної роботи № 1.

Відеозвіт: посилання на Google Drive